

Project 1

Corso di Sistemi e Architetture per Big Data A.A. 2025/26

Valeria Cardellini, Matteo Nardelli

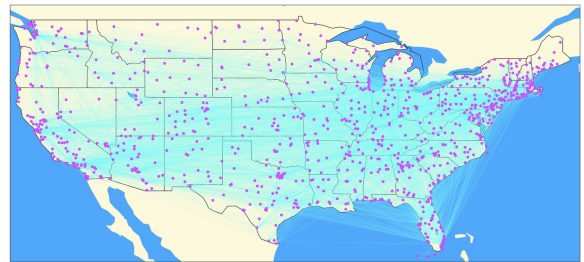
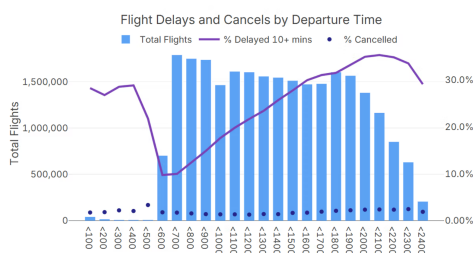
Laurea Magistrale in Ingegneria Informatica

Project delivery

- Submission deadline
 - June 5, 2026
- What to deliver
 - Link to cloud storage or repository containing project code
 - Project report composed by 4-8 pages in ACM or IEEE proceedings format
 - Presentation slides: delivered after your presentation
- Your presentation
 - June 12, 2026 (to be confirmed)
 - Max. **15 minutes** per team
- Team
 - Target: 2 students per team
 - Alternatives: 1 student or 3 students per team

Dataset

- Airline transportation
- You will analyze a dataset of **real-world airline transportation**
 - Data provided by US Department of Transportation <https://www.transtats.bts.gov/>
- Goal: analyze flight delays and cancellations



Relevant data

- Event = single flight characterized by:
 - Airline carrier
 - Flight date, including year, month, and day
 - Departure and arrival airports
 - Scheduled departure time
 - **Departure and arrival delay**, expressed in minutes; negative values indicate early flights
 - Flight status (including cancelled or diverted)
 - **Causes of delay**, including carrier, weather conditions, airport system, security, and previous aircraft

Dataset

- You will focus on flight information from January 1, 2025, to April 30, 2025
 - The dataset contains 2.2 million events
 - Preprocessed and made available on a university server
- You can download the dataset during the ingestion phase of the data pipeline

Data processing

- Use [Spark](#) framework to answer a set of queries on the dataset
 - Use RDDs/DataFrames (without using SQL)
 - Programming language: your choice
- Include the query response times on your reference platform in both the report and slides
- Save the query results in CSV files

Queries with Spark

Query 1

- Considering the airlines AA (American Airlines) and DL (Delta), aggregate the data on a monthly basis
- For each month and airline, compute:
 - Average, minimum, and maximum departure delay
 - Consider only non-cancelled flights for delay statistics
 - Cancellation rate: percentage of cancelled flights over the total number of observed flights
 - Compute the cancellation rate using all flights in each aggregation group
- Example of query output

```
# month, airline, dep-delay-mean, dep-delay-min, dep-delay-max, cancellation-rate
1, C1, 12.54, -18.00, 214.00, 1.83
2, C1, 15.02, -20.00, 231.00, 2.11
...
1, C2, 10.11, -17.00, 201.00, 1.42
2, C2, 11.98, -19.00, 223.00, 1.67
...
```

Queries with Spark

Query 1

- Generate two graphs comparing AA and DL over time:
 - Monthly average departure delay
 - Monthly cancellation rate

Queries with Spark

Query 2

- For each airline compute:
 - Total number of flights that were neither cancelled nor diverted
 - Average arrival delay
 - Average contribution of each delay component
- Considering only airlines with at least 500 non-cancelled and non-diverted flights, rank the top 10 airlines by highest average arrival delay
 - For each of these airlines, analyze the average contribution of delay causes
 - Multiple options for missing values for delay components (e.g., treated as 0, ignored), but justify your choice

Queries with Spark

Query 2

- Example of query output

```
# carrier, num_flights, arrdelay_mean, \
    carrier_delay_mean, weather_delay_mean, nas_delay_mean, \
    security_delay_mean, late_aircraft_delay_mean
F9, 12543, 18.42, 4.91, 0.62, 3.14, 0.03, 5.27
B6, 18821, 17.95, 3.88, 0.71, 3.92, 0.02, 4.86
...
```

- Generate two graphs:
 - Ranking of the top 10 airlines by average arrival delay
 - Comparing the average impact of the different delay causes for those airlines

Queries with Spark

Query 3

- Considering airlines AA (American Airlines), DL (Delta), UA (United), and WN (Southwest), aggregate the data by hour of the day, considering all 24 hourly time slots
- For each airline and each hour, consider non-cancelled flights and compute percentiles of departure delay: 25th, 50th (median), 75th, and 90th percentiles
- For each airline, compute the overall minimum and maximum departure delay across the entire dataset
- Use efficient techniques to compute percentiles on large-scale data
 - E.g., Spark's approximate percentile functions or alternative methods (e.g., t-digest, KLL sketch)

Queries with Spark

Query 3

- Example of query output

```
# airline, hour, p25, p50, p75, p90
AA, 0, 2.1, 5.3, 9.8, 18.4
AA, 1, 1.8, 4.9, 8.7, 15.2
...
DL, 0, 1.9, 4.7, 8.1, 14.9
...

# airline, min_delay, max_delay
AA, -25.0, 310.0
DL, -22.0, 285.0
...
```

- Generate graphs that compare the delay distributions across different hourly time slots for each airline

Platform and performance evaluation

- Evaluation platform
 - Standalone node: recommended approach is to use Docker Compose to orchestrate multiple containers running on the same machine
 - Alternatively, use a cloud-based Big Data processing service (e.g., Amazon EMR) with the AWS Academy grant
- Evaluate query response times on the reference platform
 - End-to-end time and optional stage breakdown (loading, pre-processing, computation, output)
 - Run multiple iterations and report average performance (optionally variance/std dev)

Optional part A: Query 4

- **Compulsory** for team composed of **3 students**
- Query 4**
- Perform a k-means clustering analysis on airlines using operational features:
 - Average departure and arrival delays
 - Cancellation rate
 - Average delay causes
 - Use feature engineering and normalization techniques before clustering
 - Select optimal number of clusters
 - Focus on top 10-15 airlines by number of flights, interpret resulting clusters, and generate visualization of clustering results

Optional part B: Spark SQL

- Use Spark SQL to solve the queries and compare the performance with respect to RDDs/Dataframes

Data acquisition and ingestion

- Which framework to ingest data into HDFS?
 - NiFi, Kafka, Pulsar, ...
- Which format to store and process data?
 - CSV, columnar format (Parquet, ORC), row format (Avro), ...
- Where to export results?
 - HBase, Redis, Kafka, MongoDB, ...

Workflow orchestration

- Optional: use a workflow orchestration tool (e.g., Airflow) to manage end-to-end data pipelines
- Typical pipeline structure
 - Data ingestion
 - Data preprocessing
 - Query execution
 - Export results

Team composition and tasks

- Send the team composition by email by **May 15, 2026**
- Mandatory tasks
 - **1 student** in the team:
 - Queries 1 and 2
 - Data acquisition and ingestion are optional (minimum requirement: HDFS)
 - **2 students** in the team:
 - Queries 1, 2, and 3
 - Data acquisition and ingestion included
 - **3 students** in the team:
 - Queries 1, 2, and 3
 - Query 4 included
 - Data acquisition and ingestion included